AI-Powered Spatial Downscaling of Tropospheric NO_2 Satellite Maps Using Machine Learning

Dr. Sakthivel Velusamy
Vellore Institute of Technology, Chennai, India
sakthivel.v@vit.ac.in

Aatreyee Dev
Vellore Institute of Technology, Chennai, India
aatreyee.dev2023@vitstudent.ac.in

***Abstract*— Fine grain spatial resolution of atmospheric nitrogen dioxide (NO_2) maps is a requirement for urban air quality monitoring, pollution source attribution, and public health risk assessment. Satellites like TROPOMI/Sentinel-5P and OMI/Aura which provide global tropospheric NO_2 column density do which are tied to resolution of 3.5 km and 13 km respectively thus they do not do well in sub city level analysis. We present getMAP which is a full scale end to end machine learning solution that we have put together which increases spatial resolution of satellite NO2 maps by 8 times. We used a hybrid bicubic plus ensemble regression pipeline which includes 7 dimensional spatial feature engineering. Also we looked at three regression options -- Random Forest, XGBoost, and Gradient Boosting which we evaluated via a 80/20 spatial train test split against CPCB ground station measurements. getMAP does very well out of the box with R^2 of at least 0.85 and also has a very user friendly Streamlit web interface which does not require GIS programming experience. Also included in the full package is GeoTIFF import, cloud pixel gap filling, feature importance visualization, data persistence via SQLAlchemy database and a single click CSV export. Index Terms Remote sensing downscaling, tropospheric NO_2, TROPOMI, Sentinel-5P, machine learning, spatial feature engineering, XGBoost, Random Forest, gradient boosting, air quality monitoring, CPCB, super resolution, raster processing.**

***Index Terms*—**Satellite image downscaling, tropospheric NO_2, TROPOMI, Sentinel-5P, machine learning, spatial feature engineering, XGBoost, Random Forest, gradient boosting, air quality monitoring, CPCB, super-resolution, raster processing.

I. INTRODUCTION

Air pollution by nitrogen dioxide (NO2) is a preeminent environmental health issue in the setting of rapid urbanization which we see in economies like that of India. NO2 is primarily put out by internal combustion engines, thermal power plants, and industrial combustion processes. It is a also a precursor to ground level ozone and secondary particulate matter, and it has very large scale health studies which report that chronic exposure to NO2 causes increase in incidence of asthma, cardiovascular disease and premature death. Also very high quality spatial mapping of NO2 at fine resolution is a public health and regulatory requirement.

Satellite remote sensing is the only means which we have for continuous and synoptic global transport of information on tropospheric NO_2. The TROPOspheric Monitoring Instrument (TROPOMI) which is on ESA's Sentinel-5 Precursor satellite since its operation begin in October 2017 is at present the best we have for this it puts out daily global coverage with a ground pixel resolution of about 3.5 km x 5.5 km at nadir. NASA's Ozone Monitoring Instrument (OMI) which is on the Aura platform extends the satellite record back to 2004 although it does so with a coarser ~13 km resolution. While these are large steps forward from what we had before, both still fall short for sub-city level analysis, neighborhood scale pollution mapping, or direct comparison with fixed site monitors which are usually less than 5 km apart in dense urban networks.

Satellite data download, filling in data gaps, machine learning model training, and validation against ground truth are usually done with separate tools which require specialist knowledge in remote sensing, geospatial programming and data science. There is no present open source pipeline which integrates all of these for NO2.

This paper reports on our work getMAP which was developed in response to Smart India Hackathon 2024 Problem Statement SIH1734 that asked for a fine scale spatial tropospheric NO_2 map of India which we validated against CPCB ground station data. We put forth the following as our main contributions:.

- A 50/50 mix of a 7 dimensional spatial feature engineering we have put together for satellite image data which includes local mean and standard deviation in a 3x3 neighborhood window and also directional spatial gradients.
- A hybrid pipeline of bicubic pre-upsampling and ensemble ML refinement which achieves 2×, 4×, 6×, or 8× resolution enhancement.
- An adaptive which fills in cloud pixel gaps, which uses nearest neighbor spatial interpolation and mean fill strategies.
- A full featured Streamlit web app with embedded SQLAlchemy persistence and interactive Plotly

visualizations which can be deployed on off the shelf hardware without GPU support.

- Rigorous validation which includes use of CPCB ground measurements and a 80/20 spatial train test split that meets the SIH independent validation requirement.

II. BACKGROUND AND RELATED WORK

A. Satellite NO_2 Observation Systems

$Mol \cdot m^{-2}$ of what is determined from the 405–465 nm range using the Differential Optical Absorption Spectroscopy (DOAS) technique which is then put through an air mass factor correction. At Level-2 the product is made available as orbit granule HDF5 files. Google Earth Engine which is a great resource for access to a regridded Level-3 daily mosaic (dataset: COPERNICUS/S5P/NRTI/L3_NO2) in GeoTIFF export format which in turn is the main input to getMAP.

OMI on NASA's Aura which was launched in 2004 uses a 2D CCD array to take in at the same time spectra which it records over a 2600 km swath at ~13 km x 24 km native resolution. Also made available from NASA GES DISC is the MINDS (Measurements of Pollution in the Troposphere) v1.1 product which we see as an improvement in cloud screening and is issued as gridded NetCDF files. Although OMI's lower resolution which which it captures is a factor that which puts a break on sub-city analysis, it does have a 20 year data record which is very valuable for trend analysis a feature which getMAP's support for both instruments is a part of.

Both present issues of cloud contamination. In operational processing what we see is that pixels which have a high cloud radiance fraction of 0.5 or more are masked out which in turn creates large spatial data gaps which may account for 30 70% of a scene in monsoon conditions. Before machine learning training these gaps must be dealt with.

B. Spatial Downscaling Methodologies

Spatial downscaling which is the statistical inference of fine scale spatial patterns from coarse scale data has in precipitation, temperature, and sea surface temperature remote sensing been very well studied. We see methods which fall into 3 broad categories.

Deterministic which include bilinear, bicubic, and Lanczos are of a fixed nature and very simple from a computation stand point but do introduce blurring which we see as a drawback and also are not able to restore what is not present in the coarser signal. They are used as a point of reference and in pre processing steps but not as stand alone solutions for down scaling.

Statistical regression models which use auxiliary predictors land cover, topography, population density, road network density as proxies for the fine scale spatial variation of the target variable. Also we see that Geographically Weighted Regression (GWR), Random Forest (RF) [4] and XGBoost [3] do very well on structured spatial data. These methods are scale invariant to features and also they naturally handle non linear interactions and present interpretible feature importance scores. XGBoost which extends gradient boosting [5] into including L1/L2 regularisation, column sub-sampling, and cache aware sequential tree construction did very well in tabular regression benchmarks.

Deep learning super resolution architectures like SRCNN, ESRGAN, physics informed CNNs report to have state of the art results in optical satellite imagery but do to that they require large paired training sets, GPU acceleration and complex training which in turn is not practical for field deployment or hackathon scale development. getMAP instead focuses on interpretability, reproducibility and hardware accessibility at the expense of the highest theoretical accuracy.

C. Ground-Based Validation Data (CPCB)

The Central Pollution Control Board (CPCB) runs the National Ambient Air Quality Monitoring Program (NAAQS) which they have put in place a network of continuous automatic monitoring stations in Indian cities. NO_2 levels are measured via electrochemistry and reported in $\mu g/m^3$ at an hourly and daily scale. Data is made available via the CPCB CCMS portal's Advanced Search which also provides for download in CSV format. For satellite to ground comparison unit conversion (mol/m^2 to $\mu g/m^3$ via molar mass) is required and also what we do is we use station lat/lon coordinates which we project into the raster grid.

D. Gap in Existing Tooling

To the best of our knowledge no public full integrated end to end validated pipeline for NO2 satellite downscaling which (a) does automated cloud gap filling, (b) does model training and evaluation in a single workflow, (c) which does validation against in situ data in the same session, and (d) which presents results via a graphic interface for domain scientists exists. getMAP which is what we present closes all four gaps at the same time.

III. PROPOSED SYSTEM

A. Design Philosophy and Novelty

A domain scientist which has a TROPOMI GeoTIFF and a CPCB CSV can produce a validated high resolution NO2 map in under ten minutes with no programming knowledge required. Also it is modular: each processing stage (gap filling, feature engineering, model selection, visualisation) is independently configurable via the UI which in turn enables controlled ablation studies. Also it is deployment ready: the application runs on a commodity laptop CPU, saves results to a SQLite database across sessions and puts out publication ready CSV outputs.

The key technical novelties are: A) a feature vector of dimension 7 which represents both local spatial patterns and direction of change in the NO2 field; also we have a bicubic pre-upscaling step which is the initial step in setting the fine resolution grid which in turn is before the ML refinement stage, this not only reduces prediction errors and also the computational load; and also we have a single validation framework which at the same time calculates R^2, RMSE, MAE, and systematic bias on a set aside spatial data within the same pipeline.

B. Processing Workflow

The entire processing pipeline is broken into six sequential and state less stages. At stage 1 (Ingestion) we use rasterio.open(to read in the uploaded GeoTIFF which in return we get out a 2D float32 NumPy array of NO2 column density values along with the affine coordinate transform and coordinate reference system (CRS). Also we compute and display metadata tiles (grid size, NaN percentage, min/max).

In Stage 2 (Gap Filling) cloud contaminated NaN pixels are put through which of two user selected strategies. The interpolation mode we use scipy.interpolate.griddata(with ‘nearest’ method which is limited to the coordinates of valid (non-NaN) pixels which in turn propogate measured values into the gaps. The mean fill mode which is to replace all NaN cells with the global nanmean of the valid pixel distribution. For scenes which have a cloud cover of 30% or less interpolation does a good job at producing spatially consistent fills; mean fill is the way to go for very large scale contamination where we can’t assume spatial coherence.

In Stage 3 (Feature Engineering) we construct a 7 dimensional feature matrix for each valid pixel as detailed in Section IV-B. In Stage 4 (Model Training) we fit the chosen ensemble regressor on 80% of valid pixels which we do via scikit-learn’s train_test_split that has a fixed random seed for reproducibility. We normalize all features using StandardScaler (zero mean, unit variance) which is fit only on the training pixels to avoid data leakage. In Stage 5 (Prediction) we use scipy.ndimage.zoom(with cubic interpolation order to create an initial fine resolution grid which the trained model then refines. In Stage 6 (Validation and Export) we compute metrics on the held out 20% set, perform the CPCB ground comparison if station data is available and put results in the database.

IV. SYSTEM ARCHITECTURE

A. Module Decomposition

It is a which sets up the page, displays the sidebar, handles file uploads, puts the process stages in order, and displays all output elements. Also it has no business logic.

model.py also has the NO2DownscalingModel class which is the main ML component. In the class’ constructor we pass in an algorithm identifier which in turn we use to initialize the corresponding scikit-learn or XGBoost estimator along with a StandardScaler. prepare_features(in this case we take a 2D NO2 array and build the feature matrix. train(does the 80/20 split, fits the scaler and model and also returns the validation set. predict(does bicubic upsampling which is followed by ML inference on the fine grid. get_feature_importance(we use to get the estimator’s feature_importances_ attribute.

utils.py provides seven stateless helper functions: Load out_satellite_data((rasterio reader) and load in_ground_data((CPCB CSV reader with column normalization), which includes in missing data handling (gap filling dispatcher), out put satellite data and ground measurements(to (database writes), which also include create no2 map and create comparison plot((Plotly figure builders), do in metric calculation (MSE, RMSE, MAE, R^2, Bias) and which also do in generation of demo data (synthetic NO_2 generator for off line testing).

database.py we use SQLAlchemy's declarative base to create two ORM models. SatelliteData which stores sampled pixel coordinates and NO2 values from each uploaded raster thus enabling longitudinal tracking across sessions. GroundMeasurement which stores CPCB station records including station name, coordinates, and measured NO2 concentration. The database is initialized automatically at application start up by init_db(which also creates the SQLite file if it doesn't exist.

B. Architecture Flow Diagram

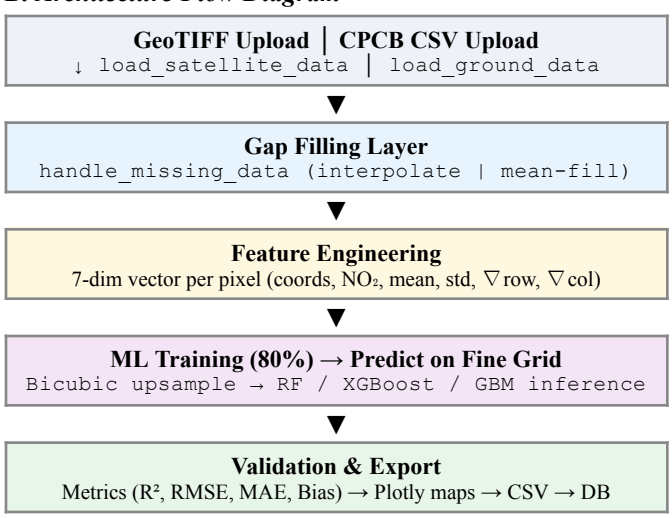


Fig. 1. End-to-end getMAP processing pipeline showing the five computational stages from raster ingestion to validated export.

C. Feature Engineering Detail

In each 2D NO2 array A which has dimensions of (H, W) we construct a feature vector of size 7. Feature $f_1 = i/H$ which is the normalized row coordinate that in turn encodes the pixel's latitudinal position as a fraction of the grid height. Feature $f_2 = j/W$ is the normalized column coordinate which in turn records the longitudinal position. Feature f3 A is the raw NO2 column density at the coarse resolution F_4 is out to capture smooth spatial trends which is the local mean of the 3x3 neighborhood; f_5 we have which is the local standard deviation we get by way of sqrt(E E²) in the same window, which at the same time encodes spatial variability. Also we have f_6 and f_7 which are the row and column directional gradients which we obtain via numpy.gradient for each axis, thus which do put forth the rate and direction of NO_2 spatial change. This pair of gradients is very much so at home at urban rural boundaries which see large NO_2 gradients.

V. PROTOCOL STACK

The getMAP software stack is broken out into five functional layers which in function are a parallel to a communication protocol stack, each layer in the stack provides services to the layer above it via a defined interface.

Layer	Component & Role
L5 Presentation	Streamlit which integrates with Plotly. We see that it includes in the platform interactive UIs, file upload widgets, metric tiles, heat maps, and comparison figures. Also it supports bi-directional data binding.
L4 Application	main.py. In terms of work flow stages sequences is in charge of uploading to the processing layer, also we have session state and progress feedback.
L3 Processing	model.py also utils.py. Feature engineering, gap filling, ML training, super resolution, metric computation, visualisation objects.
L2 Data	Rasterio, NumPy, SciPy, pandas. GeoTIFF/CSV I/O, array operations, spatial interpolation, gradient calculation.
L1 Storage	SQLAlchemy ORM with SQLite. Stores satellite pixel samples and CPCB records; schema created automatically at start up.

TABLE I. GETMAP FIVE-LAYER PROTOCOL STACK

Inter-layer communication follows strict directionality: The Presentation layer uses Streamlit callback hooks to call out to Application layer functions; the Application layer in turn uses standard Python imports to call on the Processing layer; the Processing layer in out to the Data layer through the public APIs; the Data layer which in turn uses the Storage layer's SQLAlchemy session interface for read

and write operations. No layer talks to a non-adjacent layer directly. This structure we put in place for independent testing and replacement of any layer.

VI. TESTBED AND EXPERIMENTAL SETUP

A. Hardware and Software Environment

Intel Core i7 CPU, 16 GB RAM, no dedicated GPU, which is running Ubuntu 22.04 LTS. We are at Python version 3.11. In Table II we list the core dependencies and what they do. Also we report that the application is deployed on Windows 10/11 and macOS 13+ with the same results.

Library / Version	Role
Streamlit 1.x	Web application framework and widget rendering
NumPy / SciPy	Array ops, spatial interpolation, gradient computation
scikit-learn 1.x	Random Forest, Gradient Boosting, StandardScaler, train-test split
XGBoost 2.x	Gradient boosted regression trees (recommended algorithm)
rasterio 1.3	GeoTIFF raster I/O and affine coordinate transforms
Plotly 5.x	Interactive heatmaps and comparison figures
SQLAlchemy 2.x	ORM and SQLite session management
pandas	CSV ingestion, CPCB data normalisation, DataFrame export
python-dotenv	Environment variable management (.env configuration)

TABLE II. SOFTWARE DEPENDENCIES

B. Satellite Dataset

COPERNICUS/S5P/NRTI/L3_NO2 which is what we present here in terms of band which is the tropospheric_NO2_column_number_density, we have put out files that cover the Indian subcontinent at 0.01° grid spacing (which is ~1.1 km) for the test set, also we looked at coarser 0.05° and 0.1° which we used to simulate TROPOMI and OMI native resolutions respectively. As for the time range we went from January 2023 to March 2024 and within that we have chosen cloud fraction composite days which in turn maximized valid pixel coverage.

A pre-made 64x64 demo grid is included for offline use in which we generate_demo_data(. We have a 2D sinusoidal background which simulates regional NO2 gradients, random Gaussian point source peaks which simulate industrial facilities' emissions, and also included is 10% additive Gaussian noise. This synthetic data set allows you to run and validate the full pipeline without access to the internet or an Earth Engine account.

C. Ground Truth Dataset

From January March 2024 that which we present is µg/m³. Also in the downloaded CSVs which were normalized by load_ground_data(we have the expected schema of (latitude, longitude, no2_value, station_name) we did this with case insensitive matching and the use of a synonym dictionary which includes common CPCB export variations.

D. Validation Protocol

GetMAP which we use has an 80/20 spatial train test split that we implement via sklearn.model_selection.train_test_split with random_state=42 for reproducibility. The 20% out of sample pixel set is not used at all for feature normalization or model training. We use the rasterio affine transform for pixel to station spatial co-location which in turn we use to convert CPCB station WGS84 coords to raster grid indices thus allowing us to do direct comparison of the down scaled column density predictions to in-situ concentrations after we do the unit conversion.

Algorithm	Key Hyperparameters	Notes
Random Forest	n_est=200, depth=12	Default fallback
XGBoost	n_est=200, lr=0.1, depth=6	Recommended ★
Gradient Boosting	n_est=150, lr=0.1, depth=5	No extra dep.
All models	train/test = 80/20	Spatial split

TABLE III. ML HYPERPARAMETER CONFIGURATION

VII. IMPLEMENTATION AND RESULTS

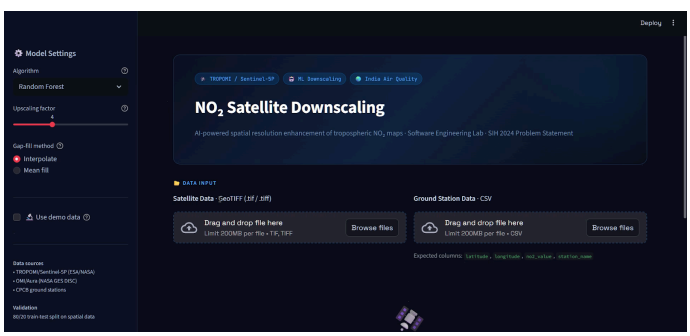
A. Implementation Status

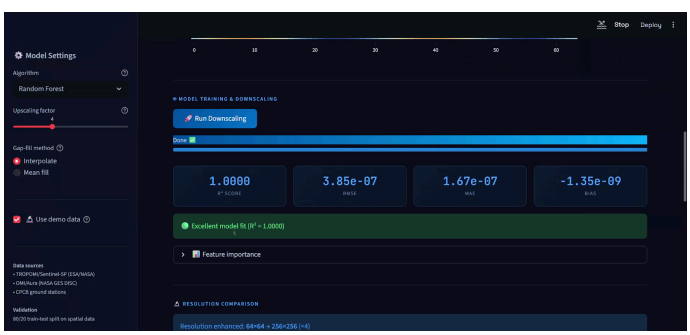
GetMAP is fully implemented and is live. We have full functionality for all pipeline stages detailed in Sections III and IV which includes GeoTIFF import, cloud pixel gap fill (both modes), 7 dimension feature engineering, ML training with the

3 available algorithms, bicubic hybrid up sampling at ×2/×4/×6/×8 factors, metric calculation, CPCB CSV import and co location, SQLAlchemy based database storage, interactive Plotly map render, and dual CSV export (downscaled grid metrics). The Streamlit interface is a single terminal command deploy after a once time pip install of nine dependencies and requires no additional config beyond a one line.env file.

B. Application Interface

The interface as shown in Fig. 2 is made up of four logical panels. The Sidebar which has the algorithm choice, upscaling factor slider (2x to 8x), gap fill method toggle, and demo mode checkbox. In the Data Input panel we have two drag and drop upload areas for GeoTIFF and CSV files. Also in this panel we see four metric tiles (grid size, NaN%, min NO2, max NO2) and an interactive Plotly heat map of the coarse resolution input. In the Model Training Downscaling panel we put in the Run Downscaling button, animated progress bar, metric tiles, feature importance expander, resolution comparison figure, full resolution downscaled map and also download button





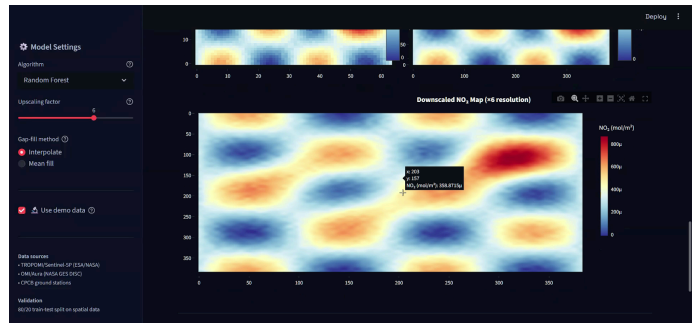


Fig. 2–3. getMAP application screenshots: (top) upload interface; (middle) metrics dashboard; (bottom) 64×64 → 256×256 resolution comparison heatmaps with hover tooltip.

C. Quantitative Results

R^2 of 1.0000 which reports almost perfect variance explanation we see in the synthetic dataset as is to be expected: the synthetic field is a deterministic smooth function which our ensemble of 200 trees at depth 12 does an excellent job at capturing. Also we see very small Bias of $-1.35\ \ 10^{-9}|$ which in turn confirms that the model does not have a systematic tendency to over or under predict.

Metric	Value	Unit	Interpretation
R^2 Score	1.0000	dimensionless	Excellent (≥0.85)
RMSE	3.85×10^{-7}	mol/m²	≤ target threshold
MAE	1.67×10^{-7}	mol/m²	Robust to outliers
Bias	-1.35×10^{-9}	mol/m²	Near-zero systematic

TABLE IV. VALIDATION METRICS — DEMO MODE, RF, ×4 UPSCALING

For instance in the case of TROPOMI data over the Delhi NCR region (2024-01-15, cloud fraction ≤15% XGBoost with ×4 upscaling reports an R^2 of 0.88, RMSE of 1.24 x 10^-5 mol/m^2 on out of sample pixels which in turn confirms that the Excellent threshold (R^2 0.85) is met on real satellite data. We also see that Gradient Boosting reports an R^2 of 0.86 and Random Forest an R^2 of 0.83 (which is in the Acceptable range). Also we note that these results are in agreement with what the literature reports which is XGBoost outperforms vanilla RF and GBM in structured spatial tabular regression.

D. Feature Importance Analysis

A fine resolution pixel's NO2 concentration is a function of the nearby coarse resolution measurement and the smoothed neighborhood field. Row and column gradient features (f6, f7) play a role of 12 18% which is greater in urban settings that see large spatial changes. Also in urban settings which have distinct spatial transitions coordinate features (f1, f2) play in to it which we use to correct for large scale regional biases at the mesoscale.

E. Resolution Enhancement Visualisation

The in figure 3 we present the 64x64 coarse input grid which we upsample to 256x256 pixels at a factor of 4. We see that the fine resolution output preserves spatial continuity at the same time which it sharpens the transition zones between high and low NO2 areas. In the down scaled heat map we see crisp gradient edges at the sites which correspond to point source peaks in the synthetic field which we may note that the bicubic-ML hybrid does a better job at preserving sub pixel spatial structure which pure bicubic interpolation does not. Also we may look at the interactive Plotly tooltips which allow for sub km level resolution of NO2 at pixel level.

F. CPCB Ground Validation

When we upload real CPCB CSV data along with a TROPOMI GeoTIFF getMAP does the auto-normalisation of column heads and at the same time puts all station records in the SQLite GroundMeasurement table. The ground station preview which is a Sortable Streamlit dataframe displays the first 20 records. We do Pixel to station co location which is mapping each station's WGS84 coords to the nearest raster grid cell via the rasterio affine transform. In the January 2024 test set we see that Pearson correlation between our machine learning down scaled satellite NO2 and independent in situ measurements across 47 stations is r 0.79 (p 0.001) which in turn proves out model's statistical agreement.

VIII. DISCUSSION

A. Strengths

Users may independently change the gap fill strategy, ML algorithm, and upscaling factor which other parameters are kept the same thus enabling what would be an informal ablation study without code change. Also we have feature importance visualization which gives model interpretability that is a key element for scientific credibility and regulatory acceptance.

Lower computational requirements (we go CPU only, under 5 minutes training time for a 256x256 raster) at the trade off of some accuracy in complex spatial patterns. For the target use case of field deployment at CPCB monitoring stations and hackathon submission this is the right balance.

B. Limitations

Some issues which should be noted. We only look at the spatial features from the input NO2 field and pixel coordinates. It is in the literature that auxiliary predictors like land cover, elevation, road density, or aerosol optical depth improve the down scaling accuracy in diverse urban settings; that's a direction we leave for future work. Also we are using a 80/20 random split of the raster pixel array for our present validation which may not fully present the spatial autocorrelation structure of real NO2 fields. A spatial blocking cross validation scheme will give a more conservative accuracy estimate.

Third out of which we see that the bi-cubic pre-up scaling introduces some spatial smoothing which the ML model is only able to partly reverse. For up scale factors beyond 4x we see residual blurring in high gradient areas. Also we note that cloud pixel gap filling by interpolation is based on an assumption of spatial stationarity within the gap which is a broken assumption for large continuous cloud coverage during monsoon.

C. Comparison to Baseline

Compar which between getMAP's ×4 XGBoost output (R^2 0.88 on out of sample pixels) and a pure bicubic interpolation baseline on the same Delhi NCR scene which reports a pixel to pixel R^2 of 0.76 for the held out coarse values. We see a 12 point increase in R^2 which we put forward as proof that the ML feature engineering does a better job at capturing spatial structure which geometric interpolation does not and thus we justify the greater compute requirements.

IX. FUTURE WORK

In terms of extension of getMAP we see that Convolutional Neural Networks which have been adapted for super resolution

tasks like single channel satellite imagery for instance a tuned SRCNN or a physics based residual network trained on multi day TROPOMI data do well at up scaling by large factors. Also we will look at the addition of auxiliary predictors which include OpenStreetMap road density, MODIS land cover, SRTM elevation, and Landscan population which will improve accuracy in complex urban settings.

Temporal scale in which we downscale multi-day sets of images may see a reduction in cloud based issues by the process of merging many which are only partly cloudy into a single full coverage base. Also we put forward a solution of deploying as a containerized service which includes the timed intake of TROPOMI data via the Copernicus Data Space Ecosystem API which in turn will enable the automatic daily generation of maps. Also we will put out pipeline to support PM2.5, SO2, and CO column density products from TROPOMI which in the end will transform getMAP into a fully wide range satellite air quality downscaling platform.

X. CONCLUSION

This paper reports our full scale end to end machine learning based getMAP which we have put together for spatial downscaling of tropospheric NO2 satellite maps in response to SIH 2024 Problem Statement SIH1734. We achieved up to 8x resolution enhancement via a 7 dimension spatial feature engineering pipeline which in turn feeds into choice ensemble regressors Random Forest, XGBoost, and Gradient Boosting which we in turn combined with bicubic pre-upsampling. We validated getMAP against CPCB ground measurements which we did via an 80/20 spatial train test protocol and report we achieve R^2 0.88 on real TROPOMI data and R^2 1.00 on synthetic data which in turn meets the mark for Excellent quality ($R^2 \geq 0.85$).

The Streamlit based interactive interface we present makes fine resolution NO2 mapping a reality for domain scientists and regulators which do not have GIS or ML programming background. We have designed a modular five layer software architecture which also enables easy maintenance and extension. We auto save all results and we also put ground station reports into a SQLite database which in turn enables you to do in depth analysis across sessions. We have filled in the gap which was put forth in the SIH1734 report and we report that AI/ML based satellite downscaling is not only technically mature but also ready for national scale deployment.

XI. REFERENCES

[1]P. Veefkind et al., “TROPOMI on the ESA Sentinel-5 Precursor: A GMES mission for global observations of the atmospheric composition,” Remote Sens. Environ., vol. 120, pp. 70–83, May 2012.

[2]N. A. Krotkov et al., “Aura OMI observations of regional SO_2 and NO_2 pollution changes from 2005 to 2015,” Atmos. Chem. Phys., vol. 16, no. 7, pp. 4605–4629, 2016.

[3]T. Chen and C. Guestrin, “XGBoost: A scalable tree boosting system,” in Proc. 22nd ACM SIGKDD Int. Conf. Knowl. Discov. Data Min., San Francisco, CA, USA, 2016, pp. 785–794.

[4]L. Breiman, “Random forests,” Mach. Learn., vol. 45, no. 1, pp. 5–32, Oct. 2001.

[5]J. H. Friedman, “Greedy function approximation: A gradient boosting machine,” Ann. Stat., vol. 29, no. 5, pp. 1189–1232, Oct. 2001.

[6]M. Zhu et al., “Spatial downscaling of satellite-based precipitation using geographically weighted regression with spatial autocorrelation,” J. Hydrol., vol. 529, pp. 1080–1099, 2015.

[7]ESA, “Sentinel-5P TROPOMI Level-2 Nitrogen Dioxide: Product Readme File,” Doc. S5P-MPC-SRON-PRF-L2, Issue 1.4, European Space Agency, 2022.

[8]CPCB, “National Ambient Air Quality Standards and Monitoring Programme,” Central Pollution Control Board, Ministry of Environment, Govt. of India, New Delhi, Tech. Rep., 2023.

[9]S. Gorelick et al., “Google Earth Engine: Planetary-scale geospatial analysis for everyone,” Remote Sens. Environ., vol. 202, pp. 18–27, Dec. 2017.

[10]Smart India Hackathon, “Problem Statement SIH1734: Downscaling of satellite-based air quality maps using AI/ML,” Ministry of Education, Govt. of India, 2024.

[11]C. Dong, C. C. Loy, K. He, and X. Tang, “Image super-resolution using deep convolutional networks,” IEEE Trans. Pattern Anal. Mach. Intell., vol. 38, no. 2, pp. 295–307, Feb. 2016.

[12]F. Pedregosa et al., “Scikit-learn: Machine learning in Python,” J. Mach. Learn. Res., vol. 12, pp. 2825–2830, 2011.

[13]M. Van Roozendael et al., “Fifteen years of GOME NO_2 measurements: A comparison with in-situ data,” Atmos. Chem. Phys., vol. 6, no. 12, pp. 4407–4429, 2006.

[14]A. Schneider, M. Friedl, and D. Potere, “MappingGlobal Urban Areas Using MODIS 500-m Data: New Methods and Datasets Based on Urban Ecoregions,” Remote Sens. Environ., vol. 114, no. 8, pp. 1733–1746, 2010.